

GmailGuard

Frontend ↔ Backend Integration Requirements

SIH26106 — AI-Powered Email Threat Detection, GeoLocation & Forensic Intelligence Platform

Purpose

This document is a handoff specification for the frontend developer. Its purpose is to ensure that every meaningful capability already present in the GmailGuard backend has a clear place in the frontend and can be consumed without duplicating backend intelligence logic.

Primary principle: the frontend is the investigation and visualization layer; the FastAPI backend remains the source of truth for parsing, analysis, correlation, scoring and reporting.

Prepared for the GmailGuard development team

1. Integration Objective

The frontend must be designed around the complete GmailGuard forensic pipeline, not only the ML phishing classifier. The user should be able to submit an email once and then navigate through the evidence produced by the backend.

End-to-end product flow

Email upload / raw email → backend parsing → header & authentication analysis → IP/network intelligence → geolocation → URL/domain intelligence → attachment analysis → ML classification → OSINT/forensic intelligence → evidence correlation → threat score → explainable findings → report.

Frontend responsibilities

- Collect the raw email or .eml file.
- Send the input to the FastAPI backend through a dedicated API service layer.
- Render the complete analysis result without recomputing security intelligence in the browser.
- Expose every available evidence category in a discoverable UI.
- Clearly distinguish PASS, FAIL, UNKNOWN, NOT OBSERVABLE and ERROR states.
- Allow the analyst to move from the high-level verdict into detailed evidence, IOCs, infrastructure and forensic context.
- Handle backend errors without falsely claiming that the entire backend is offline.

Backend responsibilities

- Parse RFC 5322 email structure.
- Extract sender, recipients, headers, body and attachments.
- Analyze authentication signals such as SPF, DKIM and DMARC.
- Extract and assess observable public IP addresses.
- Provide geolocation/infrastructure context where available.
- Extract and analyze URLs and domains.
- Perform attachment static analysis.
- Run the GmailGuard ML content classifier.
- Run OSINT/forensic domain intelligence and external intelligence providers where configured.
- Extract IOCs, correlate evidence and compute the final threat assessment.
- Generate structured forensic/report output.

2. Backend Capability → Frontend Provision Matrix

Every backend capability must have an intentional frontend destination. A feature should not be implemented in the backend and then become invisible because the frontend has no provision for it.

Backend capability	Frontend provision	Priority
Email parser	Analyze Email page + parsed-email summary + raw header viewer	Must
Header analysis	Sender/Header Intelligence section + detailed findings	Must
SPF / DKIM / DMARC	Authentication card with PASS / FAIL / UNKNOWN	Must

Backend capability	Frontend provision	Priority
Observable IP extraction	Network Infrastructure section + IP table	Must
IP reputation	Per-IP reputation card/table with source and score	Must
Geolocation	Infrastructure & Geo page/map + infrastructure details	Must
URL analysis	URL Intelligence section with per-URL findings	Must
Domain intelligence	Domain Intelligence section + investigation details	Must
Attachment analysis	Attachment Intelligence section with static findings	Must
ML classification	AI/ML Analysis card + verdict + explanation	Must
OSINT / forensic domain intelligence	Threat Intel / Forensic Evidence sections	Must
IOC extraction	IOC panel with type, value, copy and filtering	Must
Evidence correlation	Evidence Breakdown + unified findings view	Must
Threat scoring	Threat Assessment hero + score visualization	Must
Report generation	Reports page + report viewer + export actions	Must
Limitations / observability notes	Explicit limitations panel; never hide missing evidence	Must

Key requirement: do not create a single generic “AI Result” page. The frontend needs dedicated provisions for infrastructure, geolocation, threat intelligence, IOCs, forensic evidence and reporting.

3. Required Frontend Information Architecture

The navigation should make the backend capabilities discoverable.

Frontend area	What it must expose
Dashboard	Overall engine status, recent analysis summary, threat overview and shortcuts.
Analyze Email	Input/upload, analysis execution, high-level result and entry point to detailed findings.
Investigation	Unified investigation view, evidence relationships, findings and analyst-oriented drill-down.
Infrastructure & Geo	Observable IPs, mail relay infrastructure, geolocation and infrastructure context.
Threat Intel & IOCs	URLs, domains, IP reputation, external intelligence results and extracted IOCs.
Forensic Evidence	Headers, authentication evidence, forensic domain information, timeline and limitations.
Reports	Structured forensic report, executive summary and export/view actions.
Engine Status	FastAPI health, analysis availability and meaningful error state.

4. Analyze Email — Required Integration

The primary workflow must support both input modes:

- **.eml upload:** send the email file to the backend using the endpoint/schema defined by the current FastAPI API.
- **Raw email input:** send the complete RFC 5322 email string to the backend.
- Do not parse the email independently in JavaScript unless it is strictly for UI preview; backend parsing remains authoritative.

- Disable duplicate submissions while an analysis is in progress.
- Preserve the submitted email context when navigating between result sections.

The result view should surface, at minimum:

- Threat score and threat classification.
- Sender identity and domain.
- Authentication results.
- Observable IP addresses and reputation.
- Geolocation/infrastructure information.
- Extracted URLs and domain intelligence.
- Attachment findings.
- ML classification and explainability.
- OSINT/forensic findings.
- IOCs.
- Evidence correlation and reasons for the final assessment.
- Report/forensic output.
- Limitations and unavailable evidence.

5. API Integration Requirements

The current local FastAPI service is exposed at `http://127.0.0.1:8000`. The currently observed endpoints include `GET /health`, `POST /analyze` and `GET /`. The exact request/response schemas must be taken from the running FastAPI OpenAPI documentation at `/docs`; the frontend must not invent field names.

Recommended frontend API layer

Centralize all HTTP communication in one service/module, for example `api.js` or `src/api/gmailguardApi.js`. Components should consume functions such as `analyzeEmail()` and `checkHealth()` rather than making scattered fetch calls.

Conceptual API flow

```
UI input → API service → POST /analyze → normalize/display backend response → investigation UI
```

If a presentation-specific view model is needed, it should be a thin mapping layer, not a second security engine.

6. Result State Model

Security data is often incomplete. The UI must distinguish absence of evidence from evidence of safety.

State	Meaning	Frontend treatment
PASS	The backend explicitly reports a passing check.	Show as passed; do not infer that the entire email is safe.
FAIL	The backend explicitly reports a failed check.	Highlight as a finding and include its explanation.
UNKNOWN	No reliable result is available.	Show Unknown / Unavailable; do not convert to malicious or safe.
NOT OBSERVABLE	The email does not expose the requested evidence.	Explain what could not be observed and why.
ERROR	A module or pipeline operation failed.	Show module/pipeline error and preserve other successful results if the API p

State	Meaning	Frontend treatment
NOT FOUND	An intelligence provider returned no matching slow	No Match / Not Found; never label as Safe automatically.

Important example — sender IP

If the headers do not expose a sender/device IP, show **Sender IP: Not Observable**. If a public IP is visible but represents a mail relay/provider, show it as **Observable Mail Infrastructure**. Never label network geolocation as the sender's exact physical location.

7. Threat Assessment Presentation

The threat score is a backend output. The frontend should visualize it prominently but must not recalculate it from arbitrary UI-side weights.

Recommended hierarchy

- 1 Threat score.
- 2 Threat classification.
- 3 One-line executive explanation.
- 4 Top evidence contributing to the assessment.
- 5 Detailed module results below.

The current GmailGuard scoring design combines multiple evidence categories rather than relying only on ML. The frontend therefore needs a visible evidence breakdown so judges/analysts can understand how the platform is broader than a phishing classifier.

8. Explainability & Evidence Requirements

Every major backend signal should be inspectable. The frontend should make it possible to answer: “**Why did GmailGuard flag this email?**”

- ML/content finding.
- Authentication finding.
- URL/domain finding.
- IP/network finding.
- Attachment finding.
- External intelligence finding.
- Correlation-level finding.
- Any limitations that reduce certainty.

Use expandable evidence cards or a unified evidence panel. Do not force analysts to navigate through unrelated pages just to reconstruct the reason for a verdict.

9. Infrastructure & Geolocation Requirements

- Provide a dedicated infrastructure/geo view.
- Display every observable public IP returned by the backend.
- Show IP reputation alongside the IP.
- Show geolocation, ASN/provider information and infrastructure classification when returned.

- Support multiple infrastructure points if the backend returns multiple IPs.
- Clearly label relay/provider infrastructure.
- If no public IP is observable, provide an explicit no-data state.
- Never fabricate a location for visual completeness.

Geolocation is an intelligence/context feature, not an “attacker locator.” The UI wording must preserve that distinction.

10. Threat Intelligence & IOC Requirements

- Provide a dedicated IOC area for IPs, domains, URLs, email addresses and hashes when present.
- Every IOC should have a type, value and available intelligence/result fields.
- Support copy-to-clipboard for IOC values.
- Support expandable detail for individual IOCs.
- Show the intelligence source when the backend provides one.
- Represent “no match” separately from “clean.”
- Allow investigation from an IOC to related evidence when relationship data is returned.

IOC extraction, threat intelligence and forensic domain analysis should not be collapsed into one generic threat-intelligence number.

11. Relationship Graph & Forensic Timeline

If the backend returns relationship/evidence data, visualize it as relationships such as:

Sender → Domain → URL / IP → Intelligence → Geolocation / Evidence

- Graph nodes should be clickable.
- Clicking a node should reveal its backend-provided details.
- Do not create relationships that are not present in backend evidence.
- A timeline should display observed events/timestamps where available.
- If timestamps are unavailable, use an ordered evidence sequence rather than inventing times.

12. Reports

- Provide a report view that reflects the backend's structured report.
- Include executive summary, threat assessment, evidence, infrastructure, intelligence, IOCs, limitations and recommendations when returned.
- Provide export actions only for formats actually supported.
- Do not build a second independent report-generation engine in the frontend.
- Keep report content traceable to the analysis result.

13. Error Handling — Critical

The development environment has demonstrated that the FastAPI server can be healthy while an individual analysis pipeline can fail. The frontend must reflect this distinction.

Condition	Frontend should show
GET /health succeeds	Backend / FastAPI Engine: Online
POST /analyze succeeds	Analysis completed + results
POST /analyze returns 500	Backend reachable, but analysis pipeline failed
Health request fails	Backend unavailable / unreachable
Invalid email input	Input validation error; do not call it a backend outage
Partial/unknown module result	Analysis completed with limitations, if response permits

Do not show “backend is offline” whenever /analyze fails. The frontend should distinguish service availability from analysis-pipeline failure.

14. Frontend Data Architecture

Recommended conceptual state after a successful analysis:

```
analysisResult
  email / parsed email
  headers
  authentication
  ip intelligence
  geolocation
  urls
  domains / forensic domain intelligence
  attachments
  ml analysis
  osint / threat intelligence
  iocs
  evidence correlation
  threat score / classification
  explanations / findings
  report / limitations
```

This is a conceptual organization, not a claim about exact JSON field names. The implementation must follow the current FastAPI response schema.

15. No Backend Feature Left Behind — Acceptance Checklist

Before the frontend is considered integration-complete, verify each item below against the running backend.

- Can the user submit a raw RFC 5322 email?
- Can the user upload an .eml file?
- Is the complete analysis response retained in frontend state?
- Is the threat score displayed from the backend?
- Is the final classification displayed from the backend?
- Are sender and parsed header details visible?
- Are SPF, DKIM and DMARC results visible with PASS/FAIL/UNKNOWN semantics?
- Are observable public IPs visible?
- Is IP reputation visible?
- Is infrastructure/geolocation visible when available?
- Is the “sender IP not observable” case represented?

- Are URLs visible individually?
- Are domain intelligence findings visible?
- Are attachment findings visible?
- Is ML classification visible?
- Is ML explainability represented without inventing confidence values?
- Are OSINT/forensic findings visible?
- Are IOCs visible and copyable?
- Is evidence correlation visible?
- Is there an investigation/relationship view?
- Is there a forensic timeline or equivalent evidence sequence?
- Is the report view connected to backend report data?
- Are limitations visible?
- Does the UI distinguish backend offline from analysis pipeline failure?
- Does the UI avoid treating unknown/not-found as safe?
- Does the frontend avoid duplicating backend threat logic?

16. Handoff Rules for the Frontend Developer

- 1 Start from the current FastAPI OpenAPI schema at /docs and inspect the actual /analyze request and response.
- 2 Map every backend result field to a visible frontend location before building the final pages.
- 3 Keep API communication centralized in one service layer.
- 4 Keep the complete analysis response available after submission so every page can use the same investigation context.
- 5 Do not hardcode security results or threat scores.
- 6 Do not silently drop backend fields; provide them through a detail/evidence view.
- 7 Treat UNKNOWN, NOT OBSERVABLE, NOT FOUND and ERROR as distinct states.
- 8 Use backend-provided relationships/evidence for graphs and timelines rather than creating unsupported associations.
- 9 Design the UI so new threat-intelligence providers can be added later without requiring a navigation redesign.
- 10 Perform an integration walkthrough using a real .eml/raw email and verify that every returned backend capability has a visible destination.

Definition of success: a backend engineer can add or improve an analysis module without the frontend becoming unable to represent its result. The frontend should be a complete investigation surface over the backend's structured intelligence.